# Smart Water Tank Controller

Build guide v2 (corrected) · ESP32 + A02YYUW ultrasonic sensor + contactor pump control from your phone

| Item | Specification |
|---|---|
| Level sensor | DFRobot A02YYUW waterproof ultrasonic (IP67), UART 9600 baud, 3–450 cm, about ±1 cm. Mounted on the tank lid. |
| Controller | ESP32 DevKit (WROOM-32) in an IP65 box on the wall at ground level |
| Sensor cable | One outdoor CAT6 cable (up to 10 m) from tank to box. The sensor runs on 3.3 V, so no level shifter is needed. |
| Pump switching | Opto-isolated relay module → 230 V contactor → pump (single-phase up to 1 HP with MNX-9) |
| Electrical protection | Uses the pump circuit's existing MCB (and RCCB, if fitted) in the house DB; 1 A fuse F1 for the 5 V supply and the contactor control circuit; earthing |
| Firmware safety | Stop at 95 %, sensor-fault stop, max-run-time fault, dry-run (no-rise) fault, 60 s rest between starts, watchdog |
| Pump types | Single-phase submersible (through its own control box) or surface motor / monoblock pump |
| Manual override | AUTO / OFF / MAN selector on the box door |
| Phone control | MQTT over TLS (HiveMQ Cloud free tier) with IoT MQTT Panel or Home Assistant: live level, pump on/off, auto mode, fault reset |
| Total cost | **About ₹6,500 typical (range ₹5,100–8,300)**, or about ₹5,800 if no new pump cable is needed. Details in section 2. |

**What changed from v1.**

- **Sensor wires corrected:** white is TX (to GPIO16); yellow is the mode input, tied to GND. In v1 yellow was wired as TX, which would give no readings.
- The sensor runs on 3.3 V, so its TX is already at a safe level and the 1k/2k divider is removed.
- Relay module VCC is on 3.3 V and JD-VCC on 5 V, so it releases reliably when driven from a 3.3 V GPIO.
- The pump is switched by a contactor; the small relay only drives the coil.
- Added fuse F1, an AUTO/OFF/MAN selector and a pump-on lamp. The box is fed from the pump's existing MCB in the house DB. The HLK-5M05 is correctly listed as 5 V **1 A**.
- Firmware: the sensor read waits for real frames; max-run and the new dry-run fault stay latched until reset; 60 s rest between starts; watchdog code works on ESP32 core 3.x; sane Wi-Fi retry; TLS login for HiveMQ Cloud; protection against retained commands; auto mode is remembered after a reboot.
- Costs updated to 2026 Indian online prices.

## 1. How it works

The sensor on the tank lid measures the distance down to the water surface about ten times a second. The ESP32 takes the median of nine readings, converts the distance to a percentage using two calibration distances (tank full and tank empty), and decides whether the pump should run. It drives a small relay, the relay switches the coil of a contactor, and the contactor switches the pump.

Every safety decision is made inside the ESP32, so the pump stays protected if the phone, Wi-Fi or internet is down. If the ESP32 itself fails, the AUTO/OFF/MAN selector lets you run the pump by hand.

**No float switches in this build.** Overflow and dry-run protection rely entirely on the ultrasonic sensor and the firmware: the 95 % cut-off, the sensor-fault stop, the max-run limit and the dry-run check. Keep the sensor face clean and check the reading against the tank regularly. In MAN position nothing stops the pump automatically.

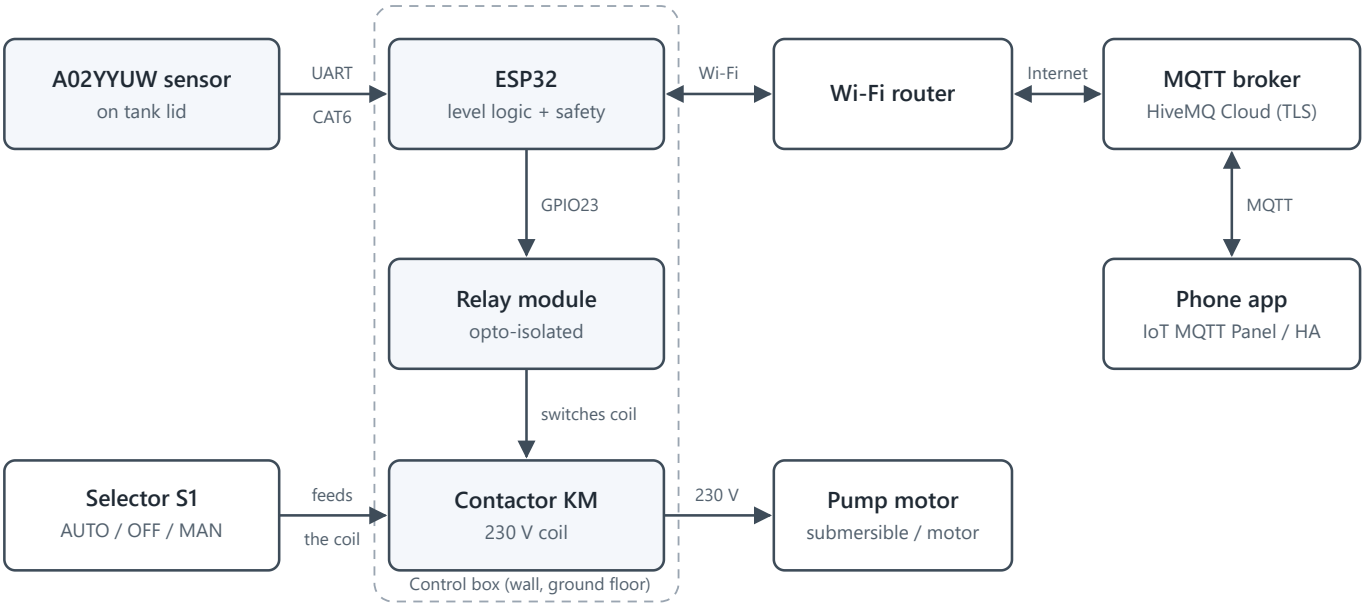



Figure 1. System block diagram.

## 2. Bill of materials and total cost

Prices are typical Indian online prices in 2026, including GST (Robu, Robocraze, Thingbits, Amazon, electrical shops). Branded switchgear (L&T, Schneider, Havells, Legrand) costs more than generic brands, but buy a branded contactor.

| # | Item | Specification / notes | Qty | Range (₹) | Typical (₹) |
|---|---|---|---|---|---|
| **A. Electronics** | | | | | |
| 1 | ESP32 DevKit | WROOM-32 board (you already have it; about ₹400–550 new) | 1 | 0 | 0 |
| 2 | Ultrasonic sensor | DFRobot A02YYUW (SEN0311), UART, IP67 | 1 | 1,500–2,000 | 1,700 |
| 3 | 5 V power module | Hi-Link HLK-5M05, 230 V AC → 5 V DC, 1 A (enough for this load) | 1 | 200–300 | 230 |
| 4 | Relay module | 1-channel, optocoupler, **with JD-VCC jumper**, 10 A contacts | 1 | 80–150 | 100 |
| 5 | Capacitors | C1 470 µF 16 V electrolytic + 100 nF ceramic | 1 set | 15–30 | 20 |
| 6 | Board and terminals | Perfboard, PCB screw terminals (4-way ×1, 2-way ×4), female headers for the ESP32 | 1 set | 150–250 | 200 |
| 7 | Fuse F1 | 1 A slow-blow fuse + DIN or panel fuse holder | 1 | 60–150 | 100 |
| | **Subtotal A** | | | **2,005–2,880** | **2,350** |
| **B. Cables and mounting** | | | | | |
| 8 | Sensor cable | Outdoor (UV-rated) CAT6, solid copper | 10 m | 300–500 | 400 |
| 9 | Tank junction box | Small IP65 box (about 100×100 mm) + 2 cable glands, for the sensor-to-CAT6 joint | 1 | 150–250 | 200 |
| 10 | Pump cable | 3-core 1.5 mm² flexible copper (skip if the pump is already wired) | ~10 m | 500–900 | 700 |
| 11 | Consumables | Ferrules, lugs, heat-shrink, cable ties, screws, sleeving | 1 lot | 150–250 | 200 |
| | **Subtotal B** | | | **1,100–1,900** | **1,500** |
| **C. Pump switching and enclosure** (the MCB and any RCCB already in your house DB are reused) | | | | | |
| 12 | Contactor KM | 9 A AC-3, 230 V AC coil (e.g. L&T MNX-9, Schneider LC1D09) | 1 | 800–1,300 | 1,000 |
| 13 | Selector switch S1 | 22 mm, 3-position (1-0-2), 2 NO contact blocks | 1 | 120–250 | 180 |
| 14 | Pilot lamp H1 | 22 mm, 230 V AC, green, "pump on" | 1 | 50–100 | 70 |

| | | | | | |
|---|---|---|---|---|---|
| 15 | Main enclosure | IP65 ABS, about 300×250×130 mm, DIN rail, 3 cable glands. Plastic, not metal, so Wi-Fi gets out. | 1 | 900–1,600 | 1,200 |
| 16 | DIN terminal blocks | L ×2, N ×2, earth ×2, end stops | 1 lot | 150–300 | 200 |
| | **Subtotal C** | | | **2,020–3,550** | **2,650** |
| | **TOTAL (A + B + C)** | | | **₹5,125–8,330** | **₹6,500** |
| | **Total if the existing pump cable can be reused (minus item 10)** | | | **₹4,625–7,430** | **₹5,800** |

| Optional / extra | Why | Approx. (₹) |
|---|---|---|
| ESP32 DevKit | Only if you need another board | 400–550 |
| Electrician | Mains section: contactor, fuse, earthing, connecting to the DB, testing | 800–2,000 |
| Pump above 1 HP (to 2 HP) | Use an 18 A contactor (MNX-18) and 2.5 $mm^2$ pump cable | +500–1,000 |
| RCCB, 2-pole 30 mA | Only if the pump circuit has no RCCB / ELCB in the DB. Strongly recommended for pumps near water. | 1,200–2,600 |

> v1 estimated ₹2,000–3,500. It left out the contactor, selector and a proper enclosure, and it underpriced the sensor, which costs ₹1,500–2,000 for the genuine DFRobot part.

## 3. Installation layout

Only the sensor (and its small junction box) goes on the tank. All electronics and mains parts sit in one box on a wall at ground level, where they are easy to reach and out of the sun.

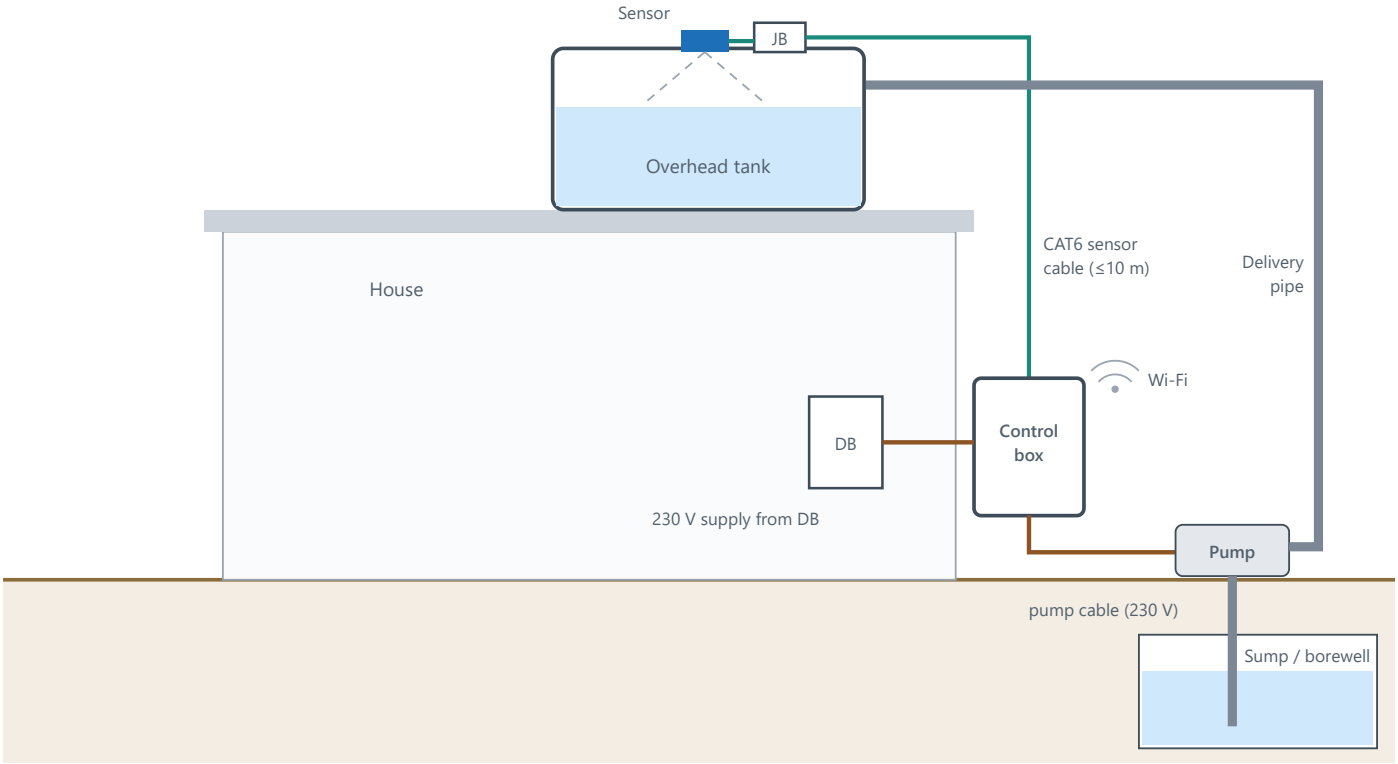



Figure 2. Physical layout. JB = small junction box where the sensor lead joins the CAT6 cable. A surface motor pump is shown; a submersible sits down in the borewell or sump instead, with its control box next to the main box.

- Run the CAT6 sensor cable at least 30 cm away from the pump cable and any mains wiring. Cross them at right angles if they have to meet.
- Bring every cable into a box from below through a cable gland, with a drip loop, so water can't run along the cable into the box.
- Keep the sensor cable at 10 m or less. Use a conduit or UV-rated cable on the roof.
- Mount the control box above any flood level, out of direct rain and sun, and within good Wi-Fi range (check signal strength on your phone at that spot).

### Sensor cable: CAT6 pair assignment

CAT6 has 8 wires in 4 twisted pairs. Doubling up power and ground keeps the voltage drop negligible, and twisting TX with a ground wire reduces noise. Join the sensor leads to the CAT6 in the tank junction box with screw terminals or solder and heat-shrink.

| CAT6 wire(s) | Sensor wire | Function | Box terminal (TB1) |
|---|---|---|---|
| Orange + white/orange | Red | VCC (3.3 V) | TB1-1 → ESP32 3V3 |
| Green | White | TX (sensor data out) | TB1-2 → ESP32 GPIO16 |
| Blue | Yellow | RX = output mode select | TB1-3 → linked to GND (real-time mode) |
| White/green + brown + white/brown | Black | GND | TB1-4 → GND |
| White/blue | — | Spare | Leave unconnected, insulated |

> **Check your sensor's wire colours.** On the DFRobot A02YYUW they are red = VCC, black = GND, **yellow = RX (mode)**, **white = TX**. Some clones differ, so check the label or datasheet that came with yours.

## 4. Low-voltage wiring (5 V / 3.3 V)

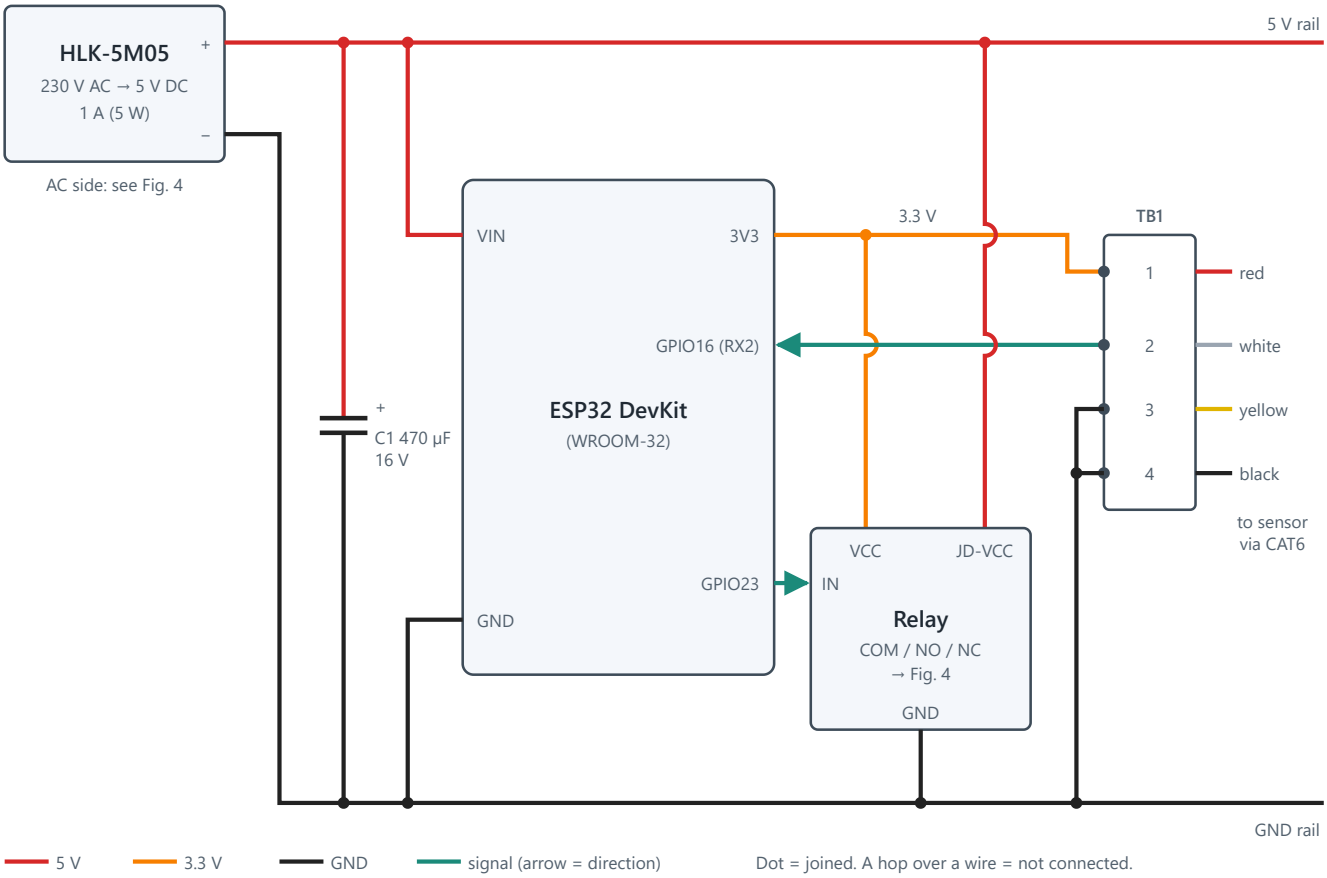



Figure 3. Low-voltage schematic. Remove the relay module's VCC–JD-VCC jumper.

| From | To | Notes |
|---|---|---|
| HLK-5M05 +Vo | 5 V rail → ESP32 VIN, relay JD-VCC | Don't plug in USB while the board is powered from VIN (unplug VIN when programming on the bench) |
| HLK-5M05 −Vo | GND rail | All grounds are common |
| C1 470 µF (+100 nF) | Across 5 V and GND, close to the ESP32 | Steadies the supply when the relay switches |
| ESP32 3V3 | TB1-1 (sensor VCC) and relay VCC | Sensor and opto draw about 15 mA total, well within the 3V3 regulator |
| TB1-2 (sensor TX, white) | ESP32 GPIO16 (RX2) | Direct; the signal is 3.3 V because the sensor runs on 3.3 V |
| TB1-3 (sensor RX/mode, yellow) | GND (link to TB1-4) | Real-time mode: a new reading every ~100 ms. Never leave it floating on a long cable. |
| ESP32 GPIO23 | Relay IN | Active LOW. GPIO23 floats during boot, so the relay stays OFF while the ESP32 starts. |
| Relay VCC / JD-VCC | 3V3 / 5 V, jumper removed | With VCC = 3V3, a HIGH GPIO puts zero current through the opto LED, so the relay always releases |

**Relay module without a JD-VCC jumper?** Buy one that says "3.3 V trigger" or "high/low level trigger", or drive the relay coil with an NPN transistor (BC547 + 1 k base resistor + flyback diode). A plain 5 V active-LOW module fed from 5 V may not switch off from a 3.3 V GPIO.

**ESP32-WROVER board?** GPIO16 and 17 are used by its PSRAM. Use another pin for the sensor (for example GPIO26) and change `SENSOR_RX` in the firmware.

## 5. Mains wiring (230 V)

**Danger: 230 V can kill.** If you are not trained, have a licensed electrician do this section. Switch off and lock out the supply before opening the box. Never work on it with wet hands or in a wet area.

**Using the existing DB.** The box is fed from the MCB that already supplies the pump in your house DB, and that MCB is also how you isolate the box. Before wiring, check two things in the DB: the MCB rating suits the pump (about 10 A C-curve up to 1 HP, 16 A for 1–1.5 HP), and whether an RCCB or ELCB (30 mA) protects that circuit. If there isn't one, adding one in the DB is the single most important safety upgrade for a pump.

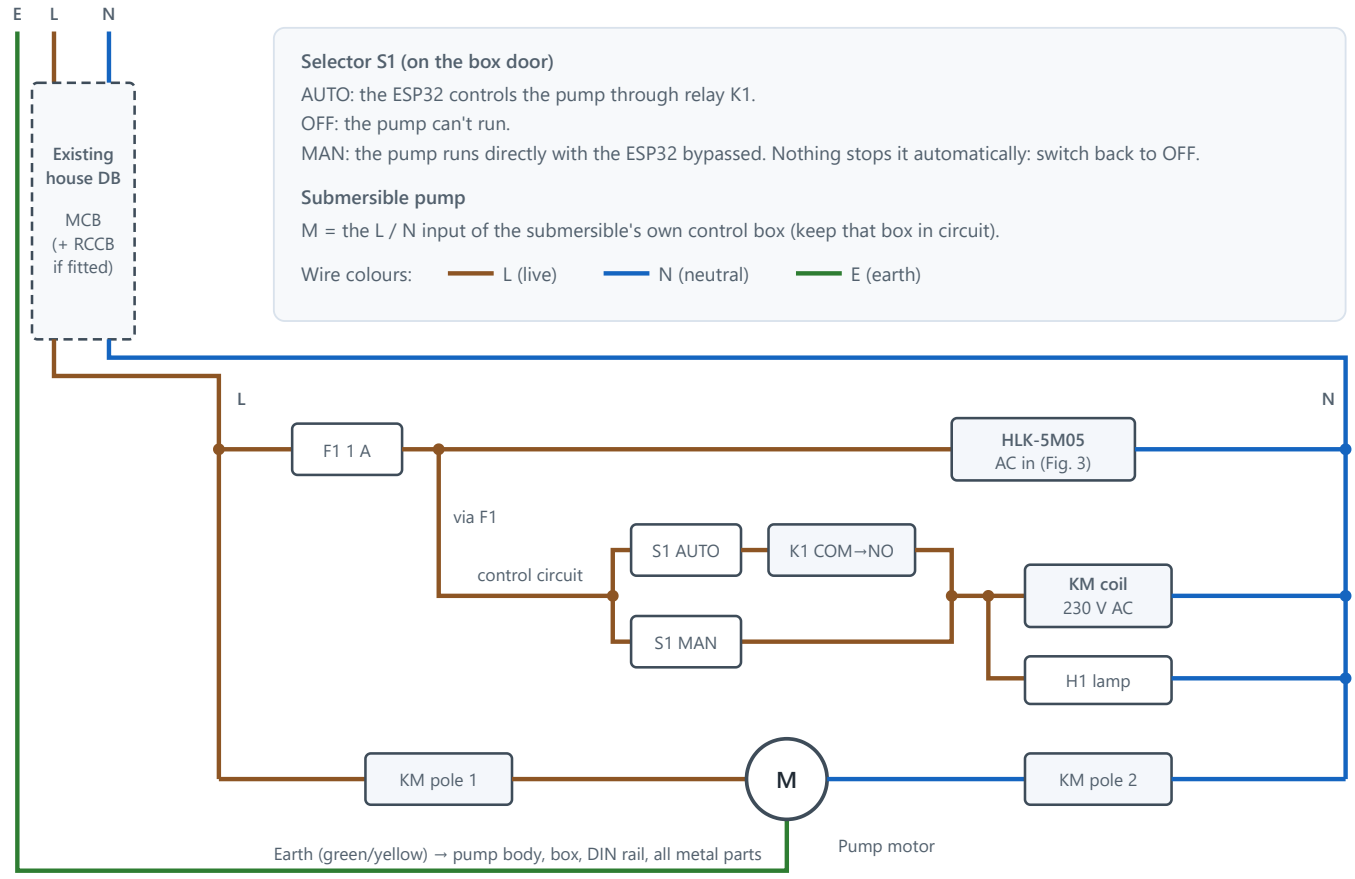



Figure 4. Mains schematic in ladder form: live rail on the left, neutral rail on the right. K1 is the contact of the relay module in Figure 3. Figure 6 (section 6) shows the same circuit as real, wire-by-wire connections.

- **Order:** the pump's existing MCB in the house DB → the box → two branches: (1) F1 fuse, which feeds both the HLK-5M05 and the control circuit (S1, relay contact, contactor coil, lamp), and (2) the contactor main contacts → pump.
- **Contactor:** use two of the MNX-9's three main poles, one for live and one for neutral, so the pump is fully isolated when it's off. MNX-9 coil terminals are A1 (from the control circuit) and A2 (to neutral).
- **Submersible pump:** single-phase submersibles come with a control box (starter and capacitors, often with an ON/OFF switch and overload). Connect the contactor's output to that control box's L and N input. Leave the control box's wiring to the pump as it is, and leave its own switch ON so the contactor does the switching.
- **Surface motor / monoblock pump:** connect the contactor's output directly to the motor's L and N terminals (its run capacitor stays as it is on the motor).
- **Pump size:** MNX-9 and 1.5 $mm^2$ cable cover single-phase pumps up to 1 HP. Many submersibles are 1–1.5 HP; for those, use an 18 A contactor (MNX-18) and 2.5 $mm^2$ cable.
- **Cables:** 1.5 $mm^2$ copper for the pump circuit (up to 1 HP), 0.75–1 $mm^2$ for the control circuit, with ferrules on every stranded end and labels on L, N and E.

- **Earth:** connect the pump body, the DIN rail and any metal in the box to the earth terminal. Check that the house earth is good; earthing (and any RCCB) relies on it.

## 6. Inside the enclosure

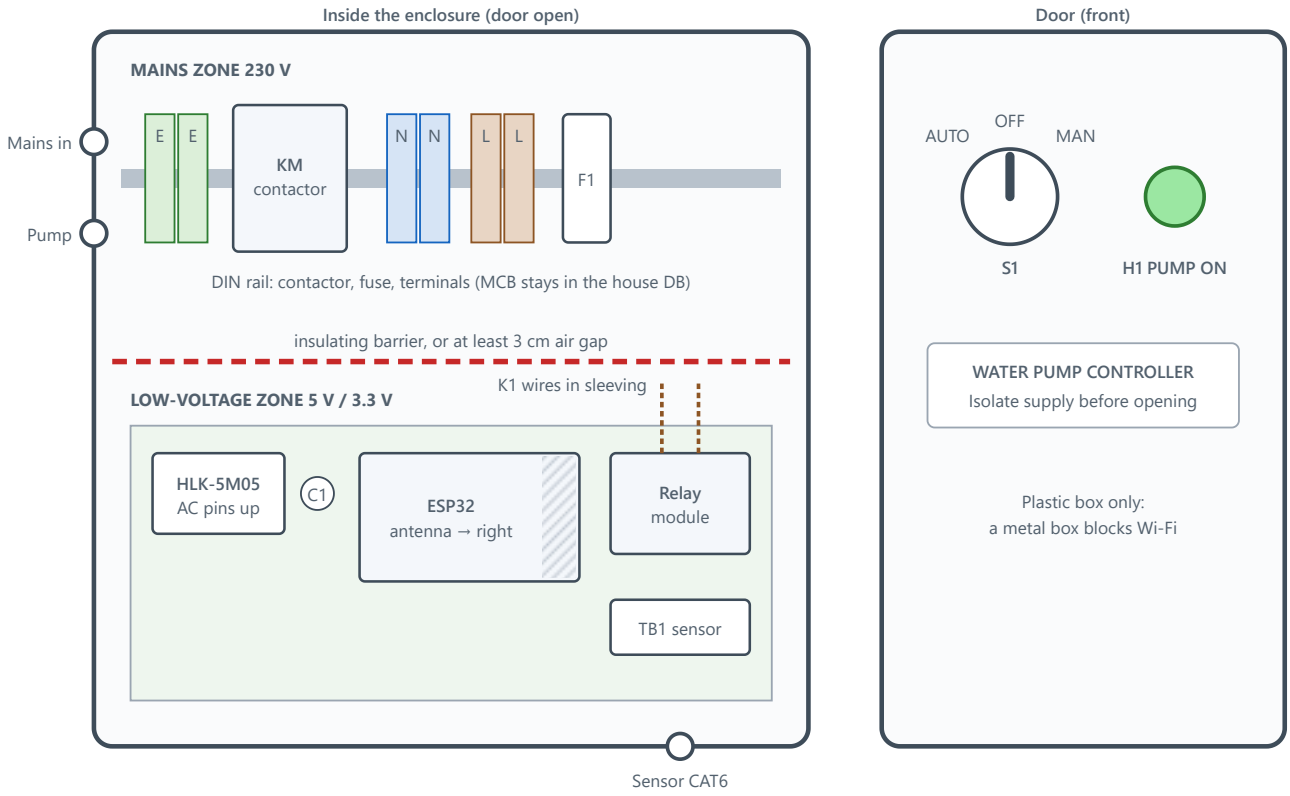



Figure 5. Suggested layout. Mains glands on the side near the DIN rail, sensor gland at the bottom near TB1, so mains and sensor wiring never share space.

## Complete wiring diagram (build from this)

Every wire in the system on one page, drawn where it physically goes. Each wire has a number in a circle; the table after the figure lists the same numbers with the exact terminals, so you can wire one row at a time and tick it off.

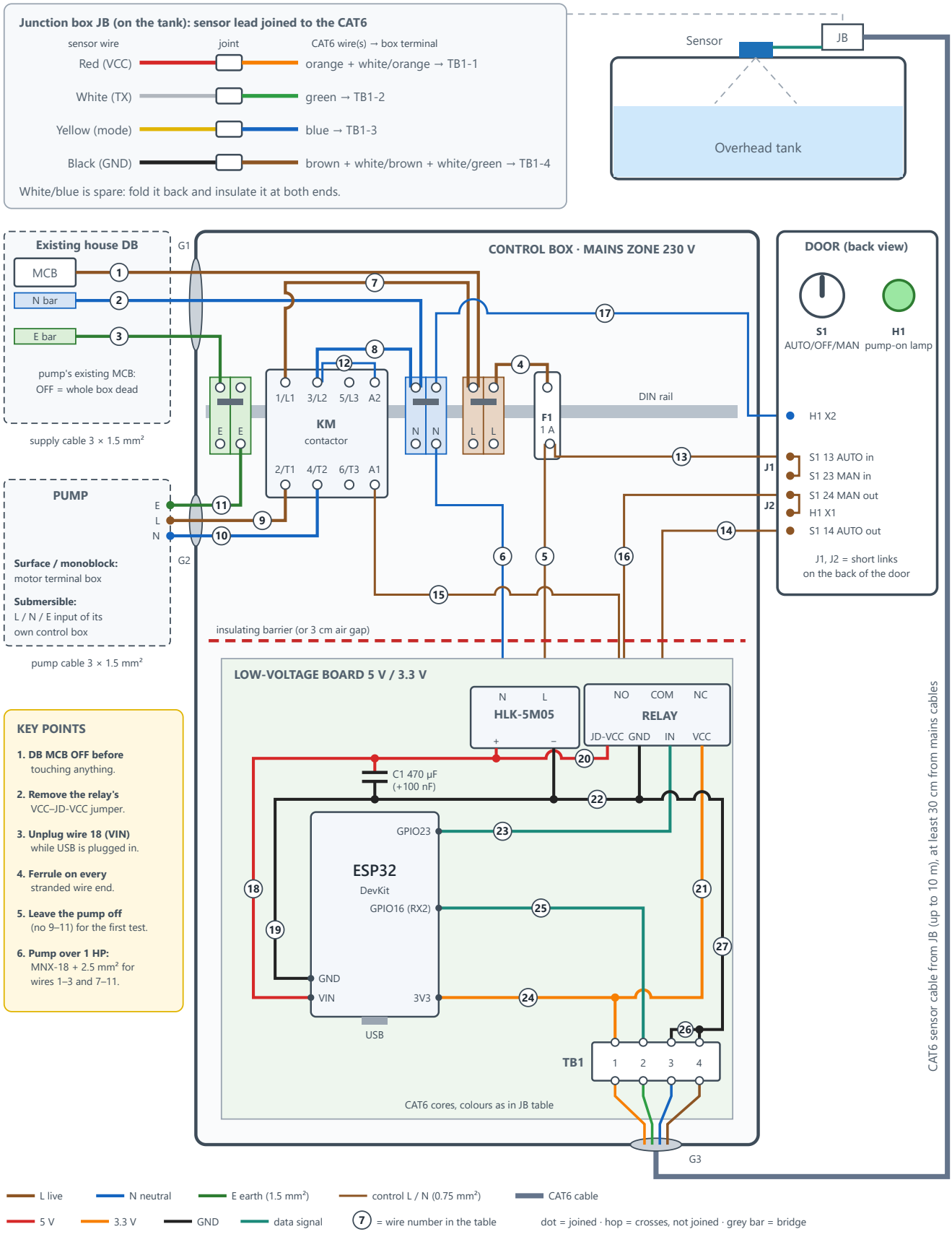



Figure 6. Complete wiring. Each pair of DIN terminals (E E, N N, L L) is linked with a plug-in bridge (grey bar), so all four screws of a pair are the same point. Where two wire ends share one screw, crimp them into one twin ferrule.

### Wire-by-wire list

Wire in this order. Switch the house DB MCB OFF first, and test with the pump disconnected (leave out wires 9–11) until section 10, step 4 passes.

| No. | From | To | Wire / notes |
|---|---|---|---|
| **A. Supply cable from the house DB (3-core 1.5 mm², enters through gland G1)** | | | |
| **1** | DB: pump MCB output | L terminal (left one), top screw | Brown (live) |
| **2** | DB: neutral bar | N terminal (left one), top screw | Blue (neutral) |
| **3** | DB: earth bar | E terminal (left one), top screw | Green/yellow (earth) |
| **B. Inside the box, mains side** | | | |
| **4** | L terminal (right one), top | F1, top | 1.5 mm² brown |
| **5** | F1, bottom | HLK-5M05 AC pin **L** | 0.75 mm² brown. Crosses the barrier. |
| **6** | N terminal (right one), bottom | HLK-5M05 AC pin **N** | 0.75 mm² blue. Crosses the barrier. |
| **7** | L terminal (left one), top | KM **1/L1** | 1.5 mm² brown. Shares the screw with wire 1 (twin ferrule). |
| **8** | N terminal (left one), top | KM **3/L2** | 1.5 mm² blue. Shares the screw with wire 2 (twin ferrule). |
| **12** | KM 3/L2 | KM **A2** (coil) | 0.75 mm² blue, short link. The coil's neutral. |
| **C. Pump cable (3-core 1.5 mm², leaves through gland G2; 2.5 mm² and MNX-18 above 1 HP)** | | | |
| **9** | KM **2/T1** | Pump L | Brown |
| **10** | KM **4/T2** | Pump N | Blue |
| **11** | E terminal (right one), bottom | Pump E (body / control-box earth) | Green/yellow. Also earth the DIN rail to this terminal. |
| **D. Control circuit (0.75 mm²; door wires as a flexible loom across the hinge)** | | | |
| **13** | F1, bottom | Door: S1 terminal **13** | Brown. Shares F1's bottom screw with wire 5. |
| **J1** | S1 terminal 13 | S1 terminal **23** | Short link on the switch, so both contacts get the same live feed |
| **14** | S1 terminal **14** (AUTO out) | Relay **COM** | Brown. Crosses the barrier. |
| **15** | Relay **NO** | KM **A1** (coil) | Brown. Crosses the barrier. |
| **16** | Relay **NO** | Door: S1 terminal **24** (MAN out) | Brown. With wire 15, two wires on the NO screw. Relay NC stays unused. |
| **J2** | S1 terminal 24 | H1 terminal **X1** | Short link on the door: the lamp lights whenever the coil is powered |
| **17** | N terminal (right one), top | Door: H1 terminal **X2** | Blue. Lamp neutral. |
| **E. Low-voltage board (0.5 mm² or jumper wire, soldered or in screw terminals; same circuit as Figure 3)** | | | |
| **18** | HLK-5M05 **+Vo** | ESP32 **VIN** | Red, 5 V |
| **19** | HLK-5M05 **−Vo** | ESP32 **GND** | Black |
| **20** | HLK-5M05 +Vo | Relay **JD-VCC** | Red, 5 V. **Remove the VCC–JD-VCC jumper** first. |
| **C1** | 5 V line (+) | GND line (−) | 470 µF 16 V, stripe (−) to GND, plus 100 nF beside it, close to the ESP32 |
| **22** | HLK-5M05 −Vo | Relay **GND** | Black |
| **23** | ESP32 **GPIO23** | Relay **IN** | Signal (active LOW) |
| **24** | ESP32 **3V3** | TB1-**1** | Orange, 3.3 V for the sensor |
| **21** | ESP32 3V3 | Relay **VCC** | Orange, 3.3 V for the opto input |
| **25** | TB1-**2** | ESP32 **GPIO16** (RX2) | Signal: sensor data into the ESP32 |
| **26** | TB1-**3** | TB1-**4** | Short link: sensor mode pin to GND (real-time mode) |
| **27** | TB1-**4** | Relay GND (GND line) | Black |
| **F. Sensor cable (CAT6 enters through gland G3; joint at the tank as in the JB table in Figure 6)** | | | |
| **TB1** | CAT6 at the box end | TB1-1 orange + white/orange · TB1-2 green · TB1-3 blue · TB1-4 brown + white/brown + white/green | White/blue spare, insulated. Same colours at both ends. |

**Before the first power-up**, with the MCB OFF, check with a multimeter (continuity): L to N on the supply terminals shows no short; earth terminal to the pump body shows continuity; F1 bottom to S1 13 and relay NO to KM A1 show continuity; TB1-3 to TB1-4 shows continuity. Then follow the test steps in section 10.

## 7. Sensor mounting and calibration

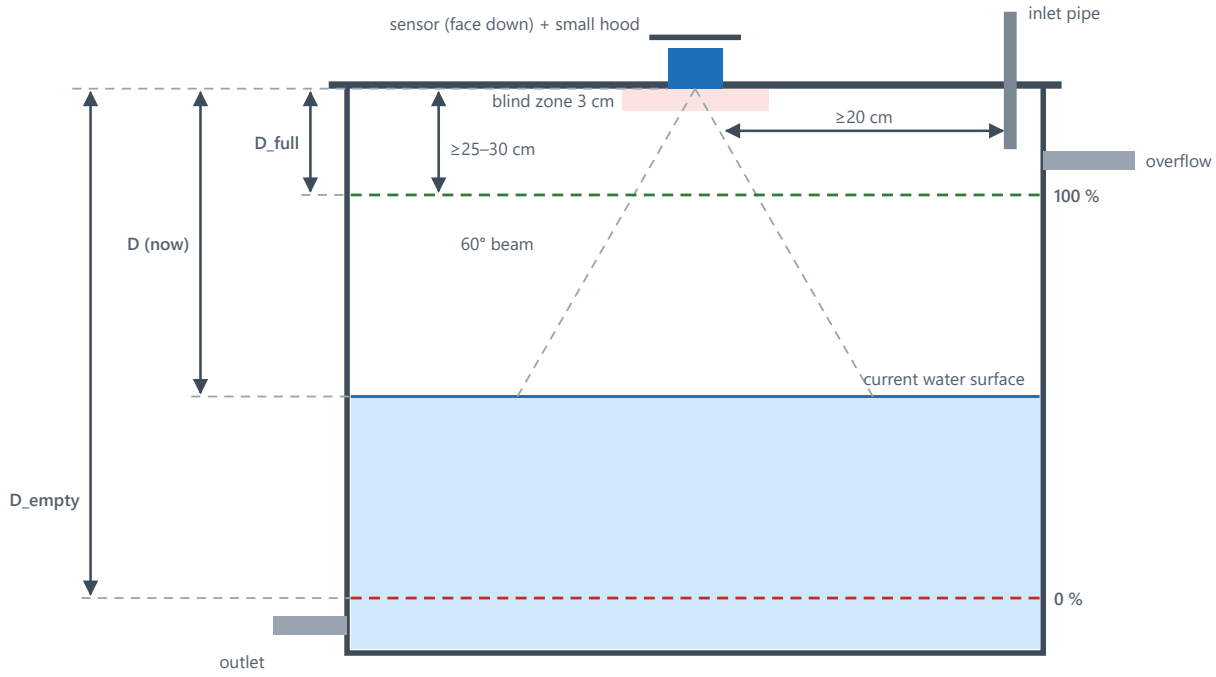



Figure 7. Side view: sensor position, blind zone and the three distances. All distances are measured from the sensor face.

- Mount the sensor **facing straight down** and level, through a hole in the lid with a cable gland. A small hood or cap above it stops condensation dripping onto the face.
- Keep the sensor face at least 25–30 cm above the 100 % line. The A02YYUW's blind zone is only 3 cm, but splashes and condensation near the top cause false readings.
- Keep it at least 20 cm from the walls, the inlet pipe and the overflow pipe. Aim it away from the splash of the inlet.
- Fix the sensor in its final position **before** calibrating. Moving it changes both calibration distances.

### Level formula

**level % = (D_empty − D) / (D_empty − D_full) × 100**, then clamp to 0–100

**Worked example.** The water can be 1.2 m deep between the 0 % and 100 % lines, and the sensor face is 30 cm above the 100 % line. So D_full = 300 mm and D_empty = 300 + 1200 = 1500 mm. If the sensor reads 900 mm: (1500 − 900) / (1500 − 300) × 100 = **50 %**.

### Calibration steps

| Step | Action |
|---|---|
| 1 | Fix the sensor in its final position, power the system, and open the Arduino Serial Monitor at 115200 baud. Each line shows `dist=… mm`. |
| 2 | **D_full:** when the tank is at the level you call 100 % (just below the overflow), note the distance. Or measure it with a tape from the sensor face down to that line. |
| 3 | **D_empty:** you don't need to drain the tank. D_empty = D_full + the depth between your 100 % line and your 0 % line (keep 0 % a little above the outlet pipe). |
| 4 | Enter both values as `D_FULL_MM` and `D_EMPTY_MM`, upload again, and check the reading against a tape at two or three levels. |

## 8. Firmware

The full sketch is in `firmware/water_tank_controller/water_tank_controller.ino` and printed in Appendix A. It compiles for the ESP32 board package 2.x and 3.x; it was test-compiled on core 3.3.8 with PubSubClient 2.8.

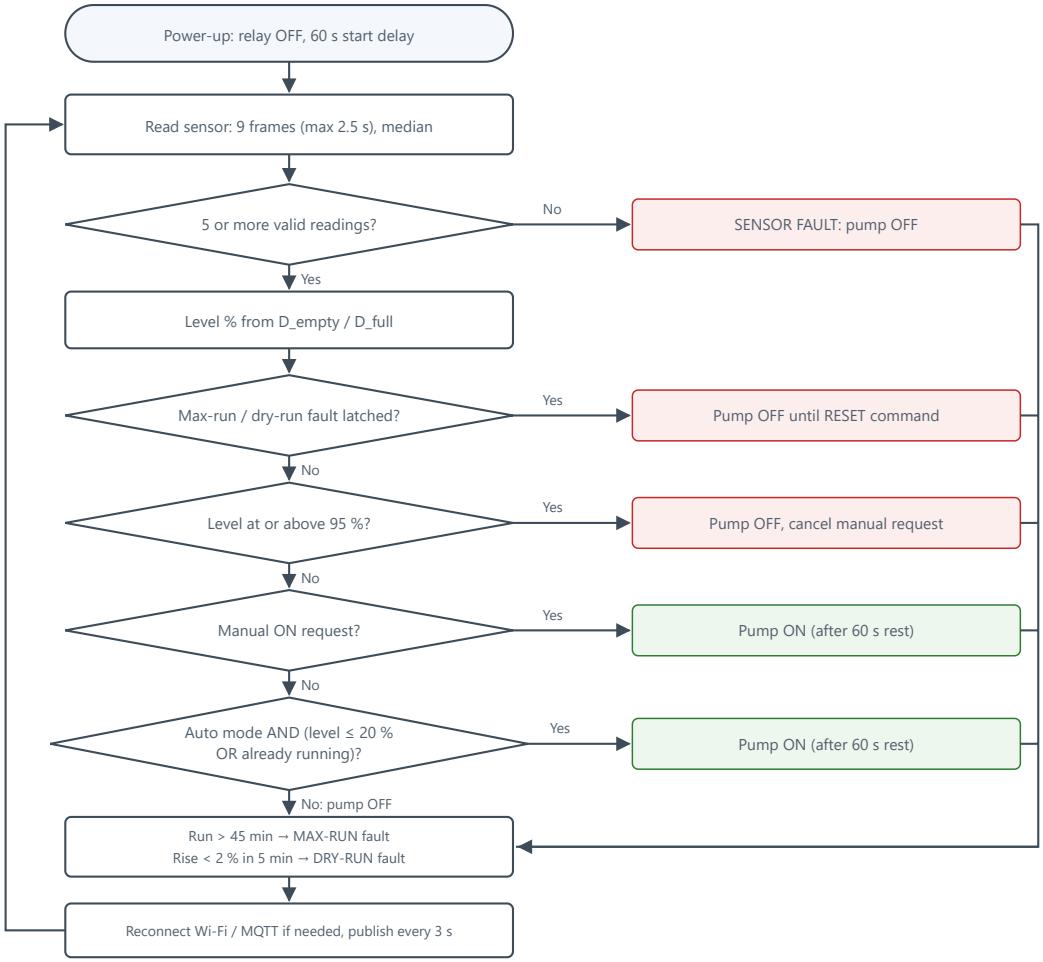



Figure 8. One pass of the main loop (about once a second).

### Safety rules

| Condition | Action |
|---|---|
| Fewer than 5 valid readings in 2.5 s | Pump OFF (sensor fault). Clears itself when readings return. |
| Level ≥ 95 % | Pump OFF and the manual request is cancelled |
| Pump has run longer than 45 min | Pump OFF, **MAX-RUN** fault latched until RESET |
| Level rose less than 2 % in any 5 min of running | Pump OFF, **DRY-RUN** fault latched until RESET |
| Auto mode, level ≤ 20 % | Pump ON until 95 % |
| Manual ON from the app | Pump ON until 95 %, a fault, or an OFF command |
| MQTT connection lost | Manual request cancelled; auto mode keeps working locally |
| Pump stopped less than 60 s ago, or just powered up | A start waits until 60 s have passed (protects the motor) |
| Firmware hangs | Watchdog resets the ESP32 after 30 s; the relay is OFF during the reset |

An OFF command from the app stops the pump. In auto mode it starts again the next time the level falls to 20 %. To keep it off, switch auto mode OFF, or turn S1 to OFF.

### Settings to adjust (top of the sketch)

| Setting | Default | How to choose it |
|---|---|---|
| `D_EMPTY_MM`, `D_FULL_MM` | 1500, 300 | From the calibration steps in section 7 |
| `ON_BELOW`, `OFF_ABOVE` | 20, 95 | Start and stop levels in %. Keep a wide gap so the pump doesn't cycle. |
| `MAX_RUN_MS` | 45 min | Time one real fill from 20 % to 95 %, then use 1.5 × that time |
| `RISE_WINDOW_MS`, `MIN_RISE` | 5 min, 2 % | In a normal fill the level must rise by at least MIN_RISE in every window. This is the only dry-run protection, so keep the window as short as your fill allows. For a very large tank with a slow pump, lengthen it. |
| `MIN_OFF_MS` | 60 s | Minimum rest between pump starts, also applied after power-up |
| `CONTROLLER_PCB` | 1 | **1** = the controller PCB (relay on GPIO32, active HIGH; see the separate PCB build sheet). **0** = the relay-module wiring in this guide (GPIO23, active LOW). |
| `RELAY_ON` / `RELAY_OFF` | set by `CONTROLLER_PCB` | With `CONTROLLER_PCB 0`, swap them only if your relay module is active-HIGH |
| `TOPIC`, `DEVICE_ID` | CHANGE_ME | Use your own unique names, for example `home/sharma/tank/` |

### MQTT topics (prefix `home/<name>/tank/`)

| Topic | Direction | Values |
|---|---|---|
| `status` | ESP32 → app (retained) | `online` / `offline` (offline is sent automatically by the broker if the ESP32 drops off) |
| `level`, `distance` | ESP32 → app (retained) | Level in % with one decimal, distance in mm; `-1` on sensor fault |
| `pump`, `auto` | ESP32 → app (retained) | `ON` / `OFF` |
| `fault` | ESP32 → app (retained) | `none`, `sensor`, `max_run`, `dry_run` |
| `pump/set` | app → ESP32 | `ON` / `OFF` |
| `auto/set` | app → ESP32 | `ON` / `OFF` (remembered after a reboot) |
| `reset` | app → ESP32 | `RESET`: clears a MAX-RUN or DRY-RUN fault |

> Send commands with **retain OFF** in the app. The firmware also clears any retained command when it connects, so an old "ON" can't restart the pump after a power cut.

### Uploading

1. Install Arduino IDE 2. In Boards Manager install **esp32 by Espressif Systems**. In Library Manager install **PubSubClient** by Nick O'Leary.
2. Open `water_tank_controller.ino` and fill in the USER SETTINGS: Wi-Fi, broker host and login, `TOPIC`, `DEVICE_ID` and the calibration values.
3. Select board **ESP32 Dev Module** and your COM port, then Upload. Unplug the 5 V VIN supply while the board is on USB.
4. Open the Serial Monitor at 115200 baud. You should see one status line about every second.

## 9. Broker and phone app setup

### HiveMQ Cloud (free)

1. Sign up at hivemq.com/cloud and create a free **Serverless** cluster.
2. Under **Access Management**, create two logins with publish and subscribe permission: `tank-device` for the ESP32 and `tank-phone` for the app.
3. Copy the cluster URL (like `abc123.s1.eu.hivemq.cloud`). The port is **8883** with TLS. Put the URL, `tank-device` and its password in the sketch, with `USE_TLS 1`.

Prefer a local broker? Run Mosquitto on a Raspberry Pi or Home Assistant, set `USE_TLS 0` and port 1883, and use a VPN (for example Tailscale) for remote access. **Never forward router ports to the ESP32.**

### IoT MQTT Panel (Android)

| Panel | Topic(s) | Settings |
|---|---|---|
| Connection | — | Host = cluster URL, port 8883, TLS/SSL on, user `tank-phone`, unique client ID |
| Gauge "Tank level" | subscribe `…/level` | Range 0–100, unit % |
| Switch "Pump" | publish `…/pump/set`, subscribe `…/pump` | Payload ON / OFF, retain off |
| Switch "Auto mode" | publish `…/auto/set`, subscribe `…/auto` | Payload ON / OFF, retain off |
| Text "Fault" and "Status" | subscribe `…/fault`, `…/status` | — |
| Button "Reset fault" | publish `…/reset` | Payload RESET, retain off |

For push notifications (level below 15 %, fault not `none`, status `offline` for more than 5 minutes), use Home Assistant's MQTT integration with automations. It also gives you history graphs and works on iPhone.

## 10. Build order and test checklist

| Phase | What to do | Pass when |
|---|---|---|
| 1. Bench sensor | ESP32 on USB, sensor through 1–2 m of CAT6 and TB1. Point it at the floor or a bucket of water. | Distance steady within a few mm. Unplug the white wire: `fault=sensor` within 3 s. |
| 2. Relay | Wire the relay module (VCC 3V3, JD-VCC 5V). Use a multimeter on COM–NO, no mains yet. | Relay stays OFF at power-up, reset and upload, and follows the app switch. |
| 3. Safety logic | Temporarily set `MAX_RUN_MS` = 2 min and `RISE_WINDOW_MS` = 1 min. Start the pump manually with the sensor aimed at a fixed target. Hold a hand close to the sensor (level above 95 %). Switch off Wi-Fi. | DRY-RUN fault after 1 min, which RESET clears. Pump OFF above 95 %. Manual run stops when MQTT drops. Restore the settings afterwards. |
| 4. Mains box | Electrician connects the box to the pump's MCB in the DB and wires F1, HLK, contactor, S1, H1 and earth. Test with the pump disconnected first. | Switching off the DB MCB kills the whole box. S1 OFF: contactor never pulls in. S1 MAN: it pulls in. S1 AUTO: it follows the relay. |
| 5. Tank | Mount the sensor, run the CAT6, calibrate. | 0 %, 50 % and 100 % agree with a tape within about 2 cm. |
| 6. Pump | Connect the pump. Let auto mode fill the tank from 20 %. Time the fill. | Starts at 20 %, stops at 95 %. Set `MAX_RUN_MS` to 1.5 × the fill time. |
| 7. Soak test | Run for one week, with S1 within reach. | No false stops or resets, readings consistent. |

## 11. Troubleshooting

| Symptom | Likely cause | Fix |
|---|---|---|
| `dist=-1`, fault "sensor" all the time | White and yellow swapped, white not on GPIO16, no 3.3 V at TB1-1, or a WROVER board | Check against the table in section 3. Measure 3.3 V at TB1-1. On WROVER, move to GPIO26. |
| Reading jumps around | Echoes from walls or the inlet pipe; splashing inlet; condensation | Move the sensor, add a hood. If it's still noisy, connect yellow to 3V3 instead of GND (processed mode, smoother but slower) |
| Stuck at one value | Water drop on the sensor face, or a bad joint in the junction box | Dry the face, fit a hood, re-make the joint |
| Always 0 % or 100 % | Calibration values wrong or swapped | D_EMPTY_MM must be larger than D_FULL_MM. Re-measure. |
| Relay never switches off | Relay VCC on 5 V with the JD-VCC jumper still fitted, or wrong RELAY_ON polarity | VCC to 3V3, JD-VCC to 5 V, jumper removed. Check RELAY_ON. |
| ESP32 resets when the contactor switches | Supply dip or switching noise | Fit C1 close to the ESP32, keep the K1 wires short and away from the ESP32, add an RC snubber (100 Ω + 0.1 µF X2) across the contactor coil |
| MQTT never connects | Wrong host, port or login; TLS setting; duplicate DEVICE_ID | Port 8883 with USE_TLS 1 for HiveMQ Cloud. Use a unique DEVICE_ID. Check the login in Access Management. |
| DRY-RUN fault although water flows | The tank is large or the pump is slow, so 2 % in 5 min is too much | Lengthen RISE_WINDOW_MS or reduce MIN_RISE |
| MAX-RUN fault on a normal fill | Limit shorter than the real fill time | Time a fill again and set 1.5 ×. If the fill really is slower than before, check for leaks, a blocked foot valve or a low sump. |
| Pump doesn't start right after power-up | The 60 s start delay | Normal. Wait one minute. |

## 12. Safety and maintenance

- With no float switches, the sensor is the only thing that sees the water. Once a month, check the app's level against the real tank level and look at the sensor face for drops, dirt or insects.
- If the pump circuit has an RCCB or ELCB in the DB, press its test button once a month. If it doesn't trip, replace it. If there is none, consider adding one.
- A submersible is cooled by the water around it, so running dry damages it within minutes. The firmware stops the pump if the tank level doesn't rise for 5 minutes. If your borewell often runs low, ask your pump dealer about the dry-run protection available for your submersible's control box.
- Every few months check the cable glands, the sensor face (condensation, insects, spider webs) and the tank junction box.
- If the electronics fail, S1 in MAN runs the pump directly. Nothing stops it automatically in MAN, so stay nearby and switch back to OFF before the tank overflows.
- Change the default passwords and topic names before going live. Anyone who knows your broker login and topics can switch your pump.

## Appendix A. Firmware listing

```
/*
  Smart Water Tank Controller v2
  ESP32 + DFRobot A02YYUW (UART) + relay module -> 230 V contactor -> pump

  Arduino IDE, board package "esp32" by Espressif (2.x or 3.x)
  Library: PubSubClient by Nick O'Leary

  Wiring (guide, Fig. 3):
    Sensor red    -> 3V3          Sensor white  -> GPIO16 (RX2)
    Sensor black  -> GND          Sensor yellow -> GND (real-time output mode)
    Relay IN      -> GPIO23       Relay VCC -> 3V3, JD-VCC -> 5V, jumper removed
*/

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <Preferences.h>
#include <esp_task_wdt.h>

// ==================== USER SETTINGS ====================
const char* WIFI_SSID = "YOUR_WIFI";
const char* WIFI_PASS = "YOUR_WIFI_PASSWORD";

#define USE_TLS 1                       // 1: HiveMQ Cloud on 8883   0: local Mosquitto on 1883
const char* MQTT_HOST = "your-cluster-id.s1.eu.hivemq.cloud";
const int   MQTT_PORT = 8883;
const char* MQTT_USER = "tank-device";
const char* MQTT_PASS = "CHANGE_ME";
const char* DEVICE_ID = "tank-esp32-CHANGE_ME";   // must be unique on the broker
#define TOPIC "home/CHANGE_ME/tank/"             // prefix of every topic

#define CONTROLLER_PCB 1                // 1: custom controller PCB (transistor driver)  0: separate relay module
#if CONTROLLER_PCB
#define RELAY_PIN 32                    // PCB socket pin 10 of the VIN row
#define RELAY_ON  HIGH                  // PCB: NPN driver, 10k pull-down keeps the relay off during boot
#define RELAY_OFF LOW
#else
#define RELAY_PIN 23
#define RELAY_ON  LOW                   // opto relay modules are usually active-LOW
#define RELAY_OFF HIGH
#endif
#define SENSOR_RX 16                    // ESP32 RX2 <- sensor WHITE wire
#define SENSOR_TX 17                    // unused: sensor YELLOW wire is tied to GND

const float D_EMPTY_MM = 1500;          // sensor reading at your 0 % level
const float D_FULL_MM  = 300;           // sensor reading at your 100 % level
const float ON_BELOW   = 20;            // auto mode starts the pump at or below this %
const float OFF_ABOVE  = 95;            // pump always stops at or above this %

const unsigned long MAX_RUN_MS     = 45UL * 60 * 1000;  // about 1.5x a normal 20 -> 95 % fill
const unsigned long RISE_WINDOW_MS = 5UL * 60 * 1000;   // dry-run check window
const float         MIN_RISE       = 2.0;               // % the level must rise in each window
const unsigned long MIN_OFF_MS     = 60UL * 1000;       // rest between starts, also after boot
const int           WDT_TIMEOUT_S  = 30;
// =======================================================

#if USE_TLS
WiFiClientSecure net;
#else
WiFiClient net;
#endif
PubSubClient mqtt(net);
Preferences prefs;

enum Latched { FAULT_NONE, FAULT_MAX_RUN, FAULT_DRY_RUN };
Latched latched = FAULT_NONE;           // cleared only by the reset command

bool autoMode = true, manualReq = false, pumpOn = false, sensorOk = false;
float level = -1, distanceMm = -1, riseRef = 0;
unsigned long pumpStart = 0, pumpStop = 0, riseRefAt = 0;
unsigned long lastPublish = 0, lastWifiTry = 0, lastMqttTry = 0;

void setPump(bool on) {
  if (on == pumpOn) return;
  if (on && millis() - pumpStop < MIN_OFF_MS) return;
```

```
  pumpOn = on;
  digitalWrite(RELAY_PIN, on ? RELAY_ON : RELAY_OFF);
  if (on) { pumpStart = riseRefAt = millis(); riseRef = level; }
  else pumpStop = millis();
  lastPublish = 0;
}

int readFrame() {                       // newest valid frame in the UART buffer, or -1
  uint8_t b[4];
  int mm = -1;
  while (Serial2.available() >= 4) {
    if (Serial2.peek() != 0xFF) { Serial2.read(); continue; }
    Serial2.readBytes(b, 4);
    if (((b[0] + b[1] + b[2]) & 0xFF) == b[3]) mm = (b[1] << 8) | b[2];
  }
  return mm;
}

float medianDistance() {                // up to 9 readings within 2.5 s, median of the valid ones
  int v[9], n = 0;
  unsigned long t0 = millis();
  while (n < 9 && millis() - t0 < 2500) {
    int d = readFrame();
    if (d > 30 && d < 4500) v[n++] = d;
    delay(20);
    esp_task_wdt_reset();
    mqtt.loop();
  }
  if (n < 5) return -1;
  for (int i = 0; i < n - 1; i++)
    for (int j = i + 1; j < n; j++)
      if (v[j] < v[i]) { int t = v[i]; v[i] = v[j]; v[j] = t; }
  return v[n / 2];
}

const char* faultName() {
  if (latched == FAULT_MAX_RUN) return "max_run";
  if (latched == FAULT_DRY_RUN) return "dry_run";
  if (!sensorOk) return "sensor";
  return "none";
}

bool wantPump() {
  if (!sensorOk || latched != FAULT_NONE) return false;
  if (level >= OFF_ABOVE) { manualReq = false; return false; }
  if (manualReq) return true;
  return autoMode && (pumpOn || level <= ON_BELOW);
}

void checkRunTimers() {
  if (!pumpOn || !sensorOk) return;
  unsigned long now = millis();
  if (now - pumpStart > MAX_RUN_MS) {
    latched = FAULT_MAX_RUN;
  } else if (now - riseRefAt > RISE_WINDOW_MS) {
    if (level - riseRef < MIN_RISE) latched = FAULT_DRY_RUN;
    riseRef = level;
    riseRefAt = now;
  }
}

void onMessage(char* topic, byte* payload, unsigned int len) {
  String t = topic, m;
  for (unsigned int i = 0; i < len; i++) m += (char)payload[i];
  m.trim();
  m.toUpperCase();
  if (m.length() == 0) return;

  if (t == TOPIC "pump/set") {
    manualReq = (m == "ON");
    if (!manualReq) setPump(false);
  } else if (t == TOPIC "auto/set") {
    bool a = (m == "ON");
    if (a != autoMode) { autoMode = a; prefs.putBool("auto", autoMode); }
  } else if (t == TOPIC "reset") {
    latched = FAULT_NONE;
  }
  lastPublish = 0;
}
```

```
void publishState() {
  String lv = sensorOk ? String(level, 1) : String("-1");
  String dv = sensorOk ? String((int)distanceMm) : String("-1");
  mqtt.publish(TOPIC "level", lv.c_str(), true);
  mqtt.publish(TOPIC "distance", dv.c_str(), true);
  mqtt.publish(TOPIC "pump", pumpOn ? "ON" : "OFF", true);
  mqtt.publish(TOPIC "auto", autoMode ? "ON" : "OFF", true);
  mqtt.publish(TOPIC "fault", faultName(), true);
}

void maintainNetwork() {
  unsigned long now = millis();
  if (WiFi.status() != WL_CONNECTED) {
    if (now - lastWifiTry > 30000) {
      lastWifiTry = now;
      WiFi.disconnect();
      WiFi.begin(WIFI_SSID, WIFI_PASS);
    }
    return;
  }
  if (mqtt.connected() || now - lastMqttTry < 5000) return;
  lastMqttTry = now;
  esp_task_wdt_reset();
  if (!mqtt.connect(DEVICE_ID, MQTT_USER, MQTT_PASS, TOPIC "status", 1, true, "offline")) return;

  mqtt.publish(TOPIC "status", "online", true);
  // Commands must never be retained, or the pump would restart on every reconnect.
  mqtt.publish(TOPIC "pump/set", "", true);
  mqtt.publish(TOPIC "auto/set", "", true);
  mqtt.publish(TOPIC "reset", "", true);
  mqtt.subscribe(TOPIC "pump/set");
  mqtt.subscribe(TOPIC "auto/set");
  mqtt.subscribe(TOPIC "reset");
  lastPublish = 0;
}

void setup() {
  digitalWrite(RELAY_PIN, RELAY_OFF);    // set the level first so the pin never drives the ON level
  pinMode(RELAY_PIN, OUTPUT);
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, SENSOR_RX, SENSOR_TX);

  prefs.begin("tank", false);
  autoMode = prefs.getBool("auto", true);

#if ESP_ARDUINO_VERSION_MAJOR >= 3
  esp_task_wdt_config_t wdt = { .timeout_ms = WDT_TIMEOUT_S * 1000, .idle_core_mask = 0, .trigger_panic = true };
  esp_task_wdt_reconfigure(&wdt);
#else
  esp_task_wdt_init(WDT_TIMEOUT_S, true);
#endif
  esp_task_wdt_add(NULL);

  WiFi.mode(WIFI_STA);
  WiFi.setAutoReconnect(true);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  lastWifiTry = millis();

#if USE_TLS
  net.setInsecure();                     // encrypted, but the broker certificate is not verified
  net.setHandshakeTimeout(8);
#endif
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
  mqtt.setCallback(onMessage);
  mqtt.setKeepAlive(30);
  mqtt.setSocketTimeout(5);
}

void loop() {
  esp_task_wdt_reset();

  distanceMm = medianDistance();
  sensorOk = distanceMm > 0;
  if (sensorOk) {
    level = (D_EMPTY_MM - distanceMm) / (D_EMPTY_MM - D_FULL_MM) * 100.0;
    if (level < 0) level = 0;
    if (level > 100) level = 100;
  } else {
    level = -1;
  }
```

```
  checkRunTimers();
  if (latched != FAULT_NONE || !mqtt.connected()) manualReq = false;
  setPump(wantPump());

  maintainNetwork();
  mqtt.loop();
  if (mqtt.connected() && millis() - lastPublish > 3000) {
    lastPublish = millis();
    publishState();
  }

  Serial.printf("dist=%d mm  level=%.1f %%  pump=%s  auto=%s  fault=%s  wifi=%d  mqtt=%d\n",
                (int)distanceMm, level, pumpOn ? "ON" : "OFF", autoMode ? "ON" : "OFF",
                faultName(), WiFi.status() == WL_CONNECTED, mqtt.connected());
}
```